

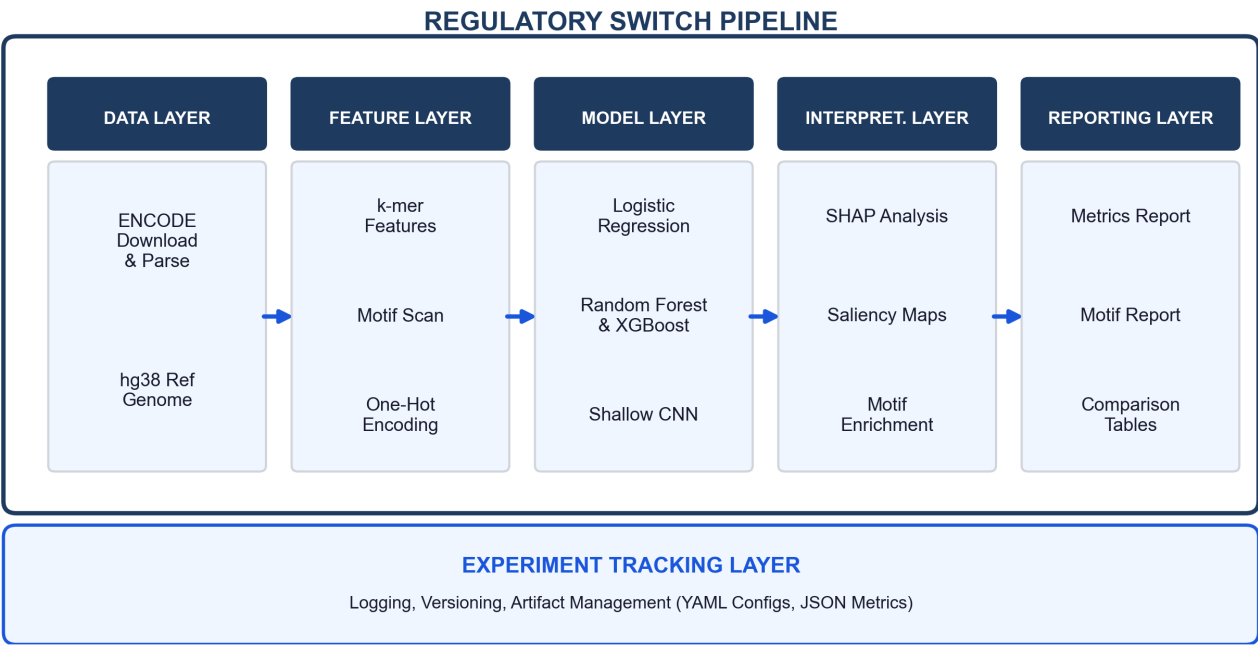
03 — Technical Design Document / Architecture Doc

Interpretable Machine Learning for Discovering Hidden Regulatory Switches in Non-Coding DNA

Field	Value
Document ID	TDD-001
Version	1.0
Status	Active
Created	2026-06-07
Related docs	PRD , Dataset Strategy , MLOps

1. System Architecture

1.1 High-Level Architecture

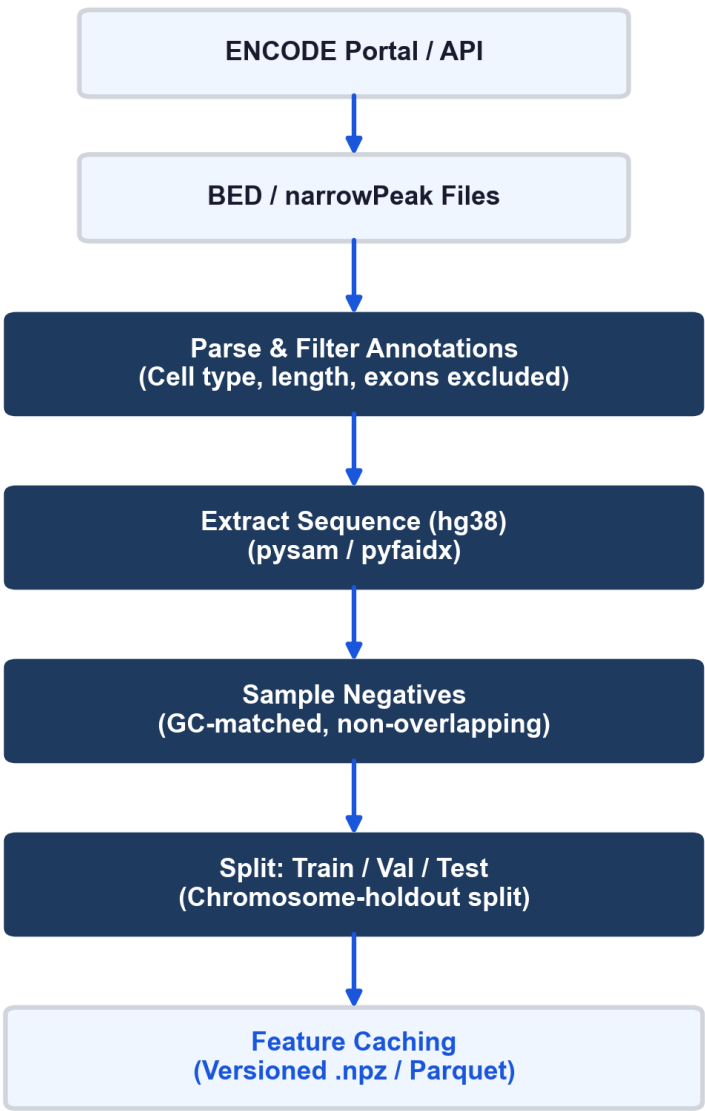


1.2 Layer Responsibilities

Layer	Responsibility	Key modules
Data Layer	Download, parse, cache ENCODE annotations; extract sequences from reference genome; generate negative sets	<code>data/download.py</code> , <code>data/parse_encode.py</code> , <code>data/sequence_extractor.py</code> , <code>data/negative_sampling.py</code>
Feature Layer	Transform raw sequences into model-ready features (k-mers, one-hot, motif scores, GC content)	<code>features/kmer.py</code> , <code>features/onehot.py</code> , <code>features/motif_scanner.py</code> , <code>features/gc_content.py</code>
Model Layer	Train, evaluate, and store classifiers across multiple model families	<code>models/logistic.py</code> , <code>models/tree_ensemble.py</code> , <code>models/cnn.py</code> , <code>models/registry.py</code>
Interpretability Layer	Extract and visualize feature importance, SHAP values, saliency maps, motif enrichment	<code>interpret/importance.py</code> , <code>interpret/shap_analysis.py</code> , <code>interpret/saliency.py</code> , <code>interpret/motif_enrichment.py</code>
Reporting Layer	Generate evaluation reports, comparison tables, publication-ready figures	<code>reports/metrics_report.py</code> , <code>reports/motif_report.py</code> , <code>reports/comparison.py</code>
Experiment Tracking Layer	Log hyperparameters, metrics, data versions; manage model artifacts	<code>tracking/logger.py</code> , <code>tracking/artifact_store.py</code>

2. Data Pipeline

2.1 Pipeline Flow



2.2 Negative Region Sampling Strategy

Negative regions require careful design to avoid trivially separable datasets.

Strategy options (implemented as configurable parameter):

Strategy	Description	Pros	Cons
Random genomic	Sample random non-overlapping regions from the same chromosomes, matched by length	Simple, fast	May sample other regulatory regions (false negatives)
GC-matched	Match GC content distribution of positive regions	Controls for compositional bias	More complex sampling
Flanking	Sample from flanking regions around positives ($\pm 2-10$ kb offset)	Controls for chromosomal context	May capture nearby regulatory activity
Exclude-known	Random sampling but exclude all known ENCODE-annotated regions	Reduces false negatives	Requires comprehensive annotation set

Default: GC-matched + exclude-known, with a 1:1 positive-to-negative ratio (configurable up to 1:5).

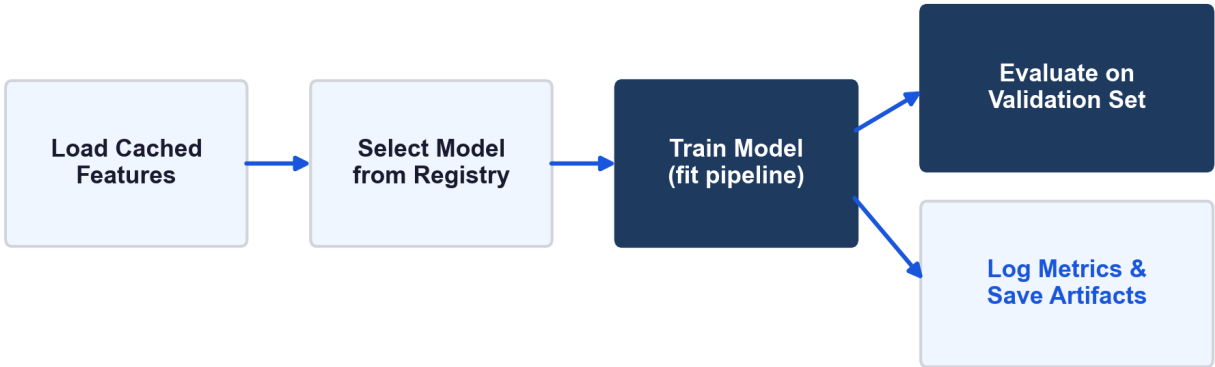
2.3 Train / Validation / Test Split

Chromosome-holdout scheme to prevent spatial leakage:

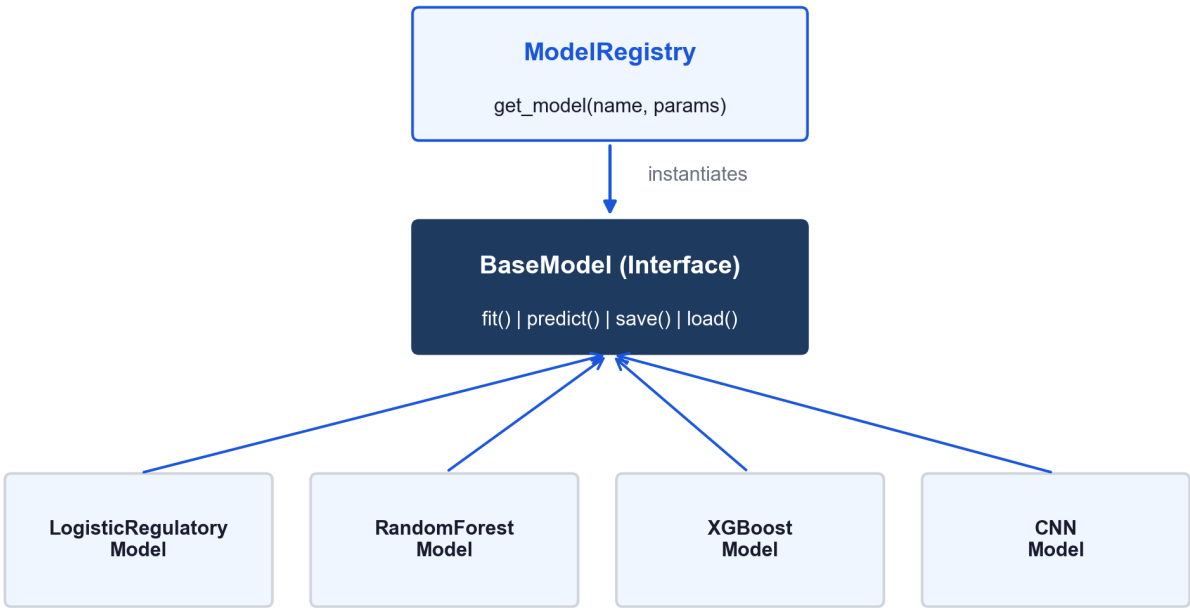
Split	Chromosomes	Rationale
Train	chr1–chr14, chrX	~70% of genome
Validation	chr15–chr18	~15% of genome
Test	chr19–chr22	~15% of genome; chr19 is gene-dense (harder test)

This ensures no region in the test set shares a chromosome with any training region, eliminating leakage from nearby regulatory elements.

3. Training Workflow



3.1 Model Registry Pattern



```
def __init__(self, concept_id: int):
    """Initialize the model registry with a concept ID"""
    self.concept_id = concept_id

class ModelRegistry:
    """ModelRegistry: A registry for storing and retrieving models"""
    def __init__(self):
        self.models = {}

    def get_model(self, name: str, params: dict) -> BaseModel:
        """Retrieve a model by name and parameters"""
        return self.models.get(name, None)

    def register_model(self, name: str, model: BaseModel, params: dict):
        """Register a new model"""
        self.models[name] = (model, params)

    def list_models(self) -> list:
        """List all registered models"""
        return list(self.models.keys())

    def delete_model(self, name: str):
        """Delete a model from the registry"""
        del self.models[name]

    def __str__(self):
        return f"ModelRegistry (concept_id: {self.concept_id})"

    def __repr__(self):
        return f"ModelRegistry (concept_id: {self.concept_id})"

    def fit(self, X_train: np.ndarray, y_train: np.ndarray):
        """Fit the model on training data"""
        self.model.fit(X_train, y_train)

    def predict_proba(self, X_test: np.ndarray) -> np.ndarray:
        """Predict probabilities for test data"""
        return self.model.predict_proba(X_test)

    def get_feature_importance(self) -> dict:
        """Get feature importance scores"""
        return self.model.get_feature_importance()

    def save(self, path: str):
        """Save the model to a file"""
        self.model.save(path)

    def load(self, path: str):
        """Load the model from a file"""
        self.model.load(path)
```

4. Validation Workflow

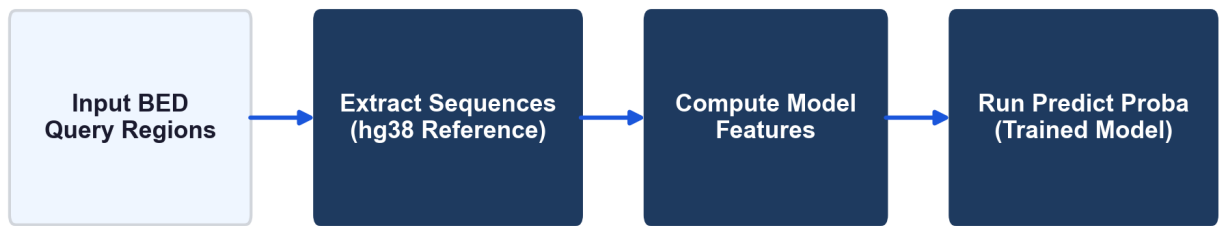
Step	Action	Output
1	Load test-set features (chromosome-holdout)	Feature matrix
2	Load trained model artifact	Model object
3	Run <code>predict_proba</code> on test set	Probability vector
4	Compute AUROC, AUPRC, F1, accuracy, precision, recall	Metrics dict
5	Generate confusion matrix	Plot
6	Run cross-validation on training chromosomes (sanity check)	CV metrics
7	Compare metrics across model families	Comparison table
8	Log all results to experiment tracker	JSON / CSV log

4.1 Stratification

- Within-split class balance is maintained via stratified sampling when doing cross-validation on training data.
- Chromosome-holdout split does not guarantee balanced classes per split; class weights or oversampling used if imbalance exceeds 1:3.

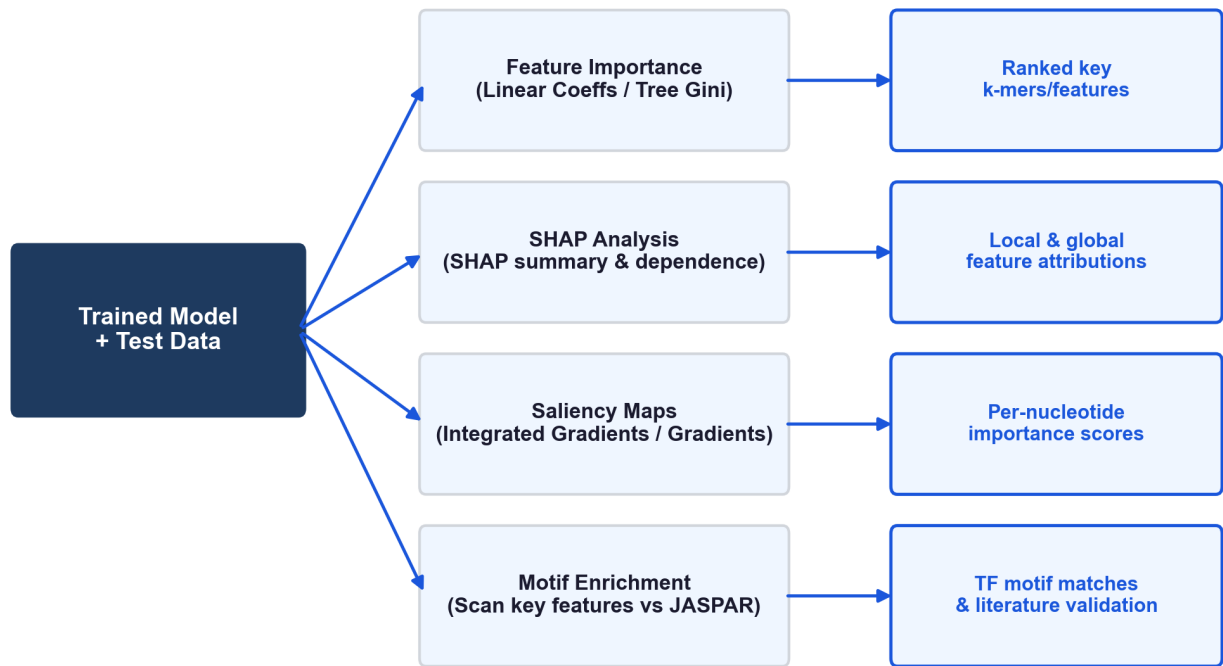
5. Inference Workflow

For batch inference on new/unseen genomic regions:



Output format: CSV/Parquet with columns: `chrom`, `start`, `end`, `prediction`, `probability`, `model_name`, `model_version`.

6. Interpretability Workflow

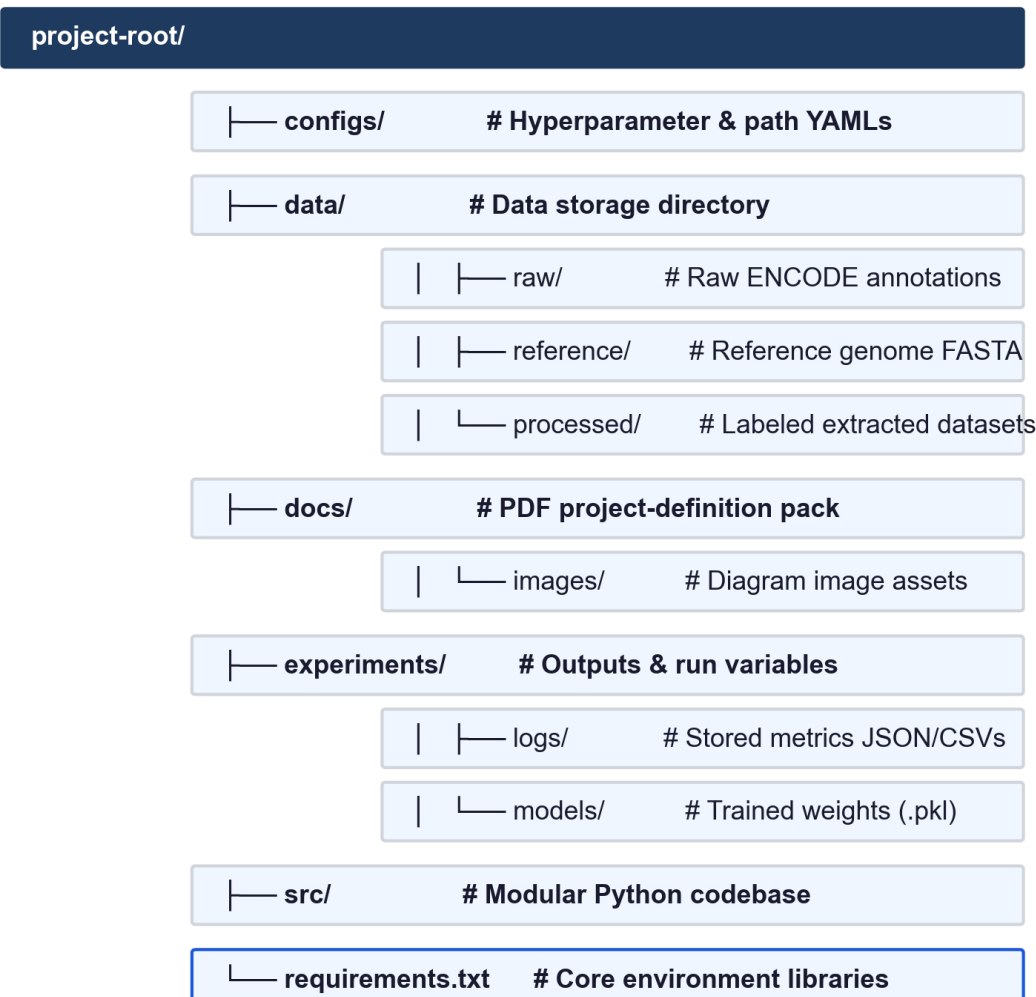


6.1 Interpretability Output Artifacts

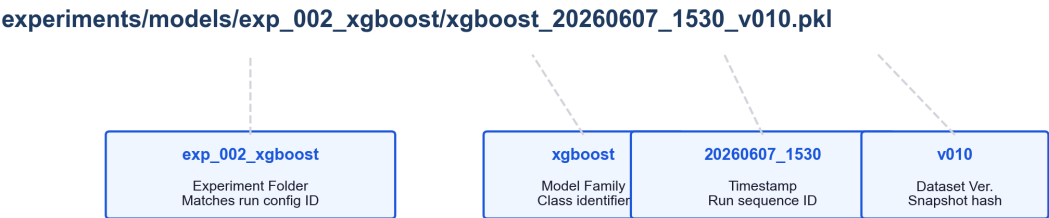
Artifact	Format	Model families
Feature importance table	CSV	All
Feature importance bar chart	PNG/SVG	All
SHAP summary plot	PNG/SVG	Tree-based, Linear
SHAP waterfall (per-sample)	PNG/SVG	Tree-based, Linear
Saliency heatmap	PNG/SVG	CNN
Integrated gradients map	PNG/SVG	CNN
Motif enrichment table	CSV	All (post-processing)
Motif logo plots	PNG/SVG	All (post-processing)

7. Storage and Artifact Strategy

7.1 Directory Structure



7.2 Artifact Naming Convention



```
experiments/models/{experiment_name}/{model_family}_{timestamp}_{dataset_version}.pkl
experiments/logs/{experiment_name}/{model_family}_{timestamp}.json
experiments/figures/{experiment_name}/{plot_type}_{model_family}_{timestamp}.png
```

7.3 Kaggle Dataset Integration

On Kaggle, datasets are mounted at `/kaggle/input/`. The project should:

- 1. Package curated ENCODE subsets as Kaggle datasets (private or public).
- 2. Reference genome chromosomes as a separate Kaggle dataset.
- 3. Use `/kaggle/working/` for outputs.
- 4. Kaggle notebooks should import `src/` modules via a utility dataset containing the codebase, or use `%%writefile` magic to define modules inline.

8. Notebook vs. Modular Code Structure

8.1 Dual-Mode Strategy

Context	Approach
Kaggle execution	Self-contained notebooks that inline critical functions or import from a bundled <code>src/</code> package uploaded as a Kaggle dataset
Local development	Standard Python package (<code>src/</code>) with notebooks for exploration only; scripts for production runs
Testing	<code>pytest</code> -based tests in <code>tests/</code> ; run locally, not on Kaggle

8.2 Notebook Hygiene Rules

- 1. Every notebook starts with a header cell: experiment name, date, purpose, data version.
- 2. Notebooks must not define critical logic inline — use `src/` modules.
- 3. Exception: Kaggle-specific notebooks may inline code when necessary, but must clearly mark inlined sections.
- 4. Notebooks must be runnable top-to-bottom without manual intervention.
- 5. Output cells should be cleared before committing to version control.

9. Recommended Stack

9.1 Core Dependencies

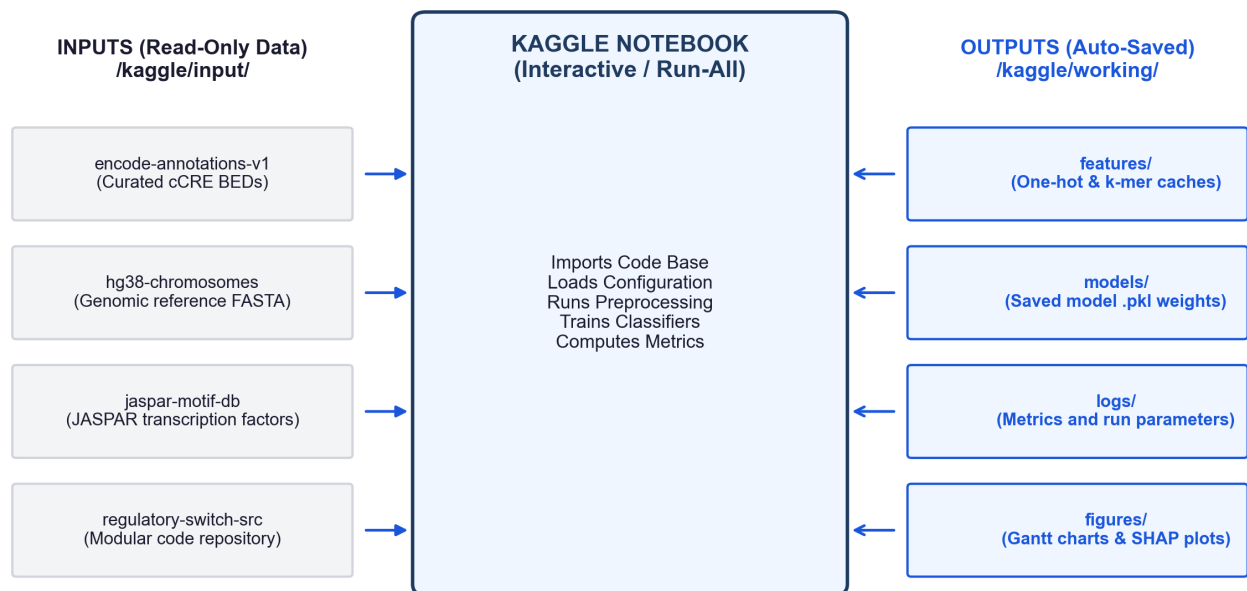
Category	Package	Version constraint	Purpose
Data	<code>pandas</code>	≥ 1.5	Tabular data handling
Data	<code>numpy</code>	≥ 1.24	Numerical computation
Data	<code>pysam</code>	≥ 0.21	FASTA/BAM file access
Data	<code>pybedtools</code>	≥ 0.9	BED file operations
Data	<code>biopython</code>	≥ 1.81	Sequence utilities
ML	<code>scikit-learn</code>	≥ 1.3	Classical ML models, metrics, preprocessing
ML	<code>xgboost</code>	≥ 1.7	Gradient boosting
ML	<code>lightgbm</code>	≥ 4.0	Alternative gradient boosting
DL	<code>torch</code>	≥ 2.0	CNN and attention models
Interpret	<code>shap</code>	≥ 0.42	SHAP values
Interpret	<code>captum</code>	≥ 0.6	PyTorch interpretability (saliency, IG)
Viz	<code>matplotlib</code>	≥ 3.7	Plotting
Viz	<code>seaborn</code>	≥ 0.12	Statistical visualization
Tracking	<code>pyyaml</code>	≥ 6.0	Configuration files
Motif	<code>logomaker</code>	≥ 0.8	Sequence logo visualization

9.2 Optional / Later-Stage Dependencies

Package	Purpose	When to add
<code>transformers</code> (HuggingFace)	Pretrained genomic model loading	Stage 3
<code>wandb</code>	Experiment tracking UI	When local tracking becomes insufficient
<code>mlflow</code>	Alternative experiment tracking	When local tracking becomes insufficient
<code>gradio</code>	Simple demo UI	When a public demo is desired
<code>onnx</code> / <code>onnxruntime</code>	Model export for deployment	If deployment is ever needed

10. Kaggle-Oriented Execution Pattern

10.1 Recommended Kaggle Setup



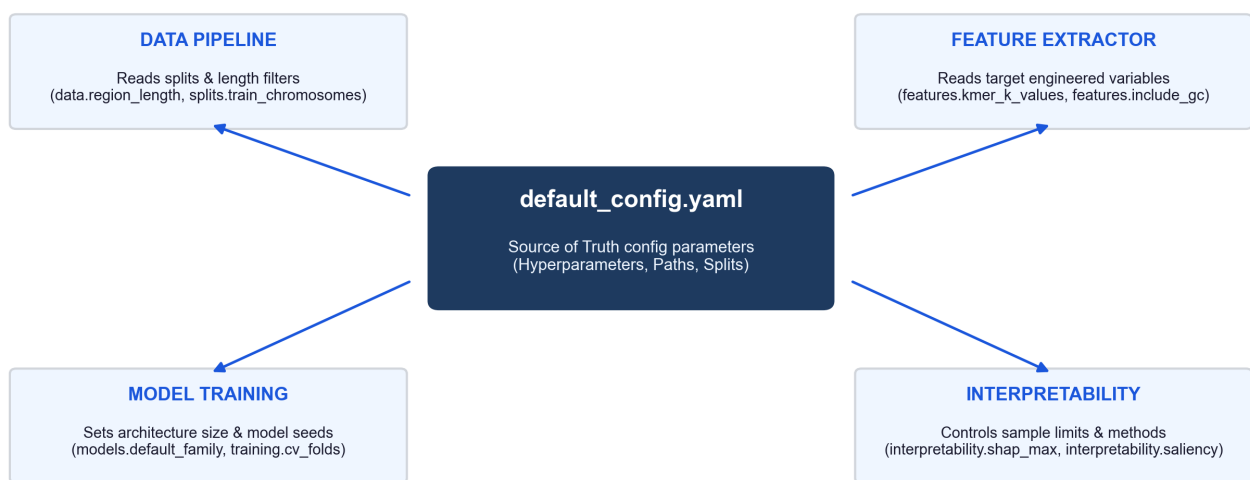
10.2 Kaggle Session Budget

Task	Estimated time	GPU needed?
Data parsing + sequence extraction	20–40 min	No
k-mer feature computation (100k regions, k≤6)	10–30 min	No (CPU)
One-hot encoding (100k regions × 1 kb)	5–10 min	No
Logistic regression training	1–5 min	No
Random forest / XGBoost training	5–20 min	No
Shallow CNN training (50 epochs)	30–90 min	Yes
SHAP computation (tree-based)	10–30 min	No
Saliency maps (CNN, 1000 samples)	5–15 min	Yes
Full pipeline	2–4 hours	Partially

10.3 Memory Management on Kaggle

- Load reference genome chromosomes individually, not the full FASTA.
- Use `float32` for features, not `float64`.
- Compute features in batches and write to disk, then load batch-by-batch.
- Use sparse matrices for high-dimensional k-mer features ($k \geq 5$).
- Delete intermediate variables and call `gc.collect()` after major operations.
- For CNN: use `DataLoader` with reasonable `batch_size` (64–256) to manage GPU memory.

11. Configuration Management



11.1 Default Configuration File



12. Security and Access Considerations

- All data is public. No authentication or access control is required for data.
 - Kaggle API keys should be stored in Kaggle Secrets, not hardcoded.
 - No PHI (Protected Health Information) is involved.
 - Model artifacts do not contain sensitive information (they contain learned weights, not raw sequences).
-

13. Future Architecture Extensions (Later Phases Only)

Note: The following are documented for planning purposes only. They are not part of the initial build.

Extension	Architecture impact	Phase
Pretrained model embeddings	New feature extraction pathway using HuggingFace transformers	Stage 3
Multi-omics integration	Additional data loaders (expression, methylation)	Stage 4
Web-based demo (Gradio)	New serving layer; model export to ONNX	Post-Stage 3
Distributed training	Migration from Kaggle to cloud compute	Only if funded
Database-backed experiment tracking	Replace file-based logging with MLflow/W&B server	When team size > 5

End of Document 03 — Technical Design Document
Next: [04 — Research Design Document](#)